شرح سطر بسطر ٧ — شاشة 02: إنشاء الملف، الخطوة 1 (الاسم والصف)

مشروع رِزمة · Flutter · كل سطر مع سبب كتابته بهذه الطريقة

الإطار في `اختيار الصف 02` :Figma. تجمع من الطالب شيئين فقط: اسمه وصفّه (1–12). لا تُنشئ شيئًا في القاعدة بعد؛ تحفظ ما جمعته في «مسودة» وتنتقل للخطوة 2 ثم يُنشأ الملف كله دفعة واحدة.

هذه الشاشة أول شاشة **تفاعلية** (حقل نص وصف مختار وزر يتفعّل ويتعطّل) لذلك تستعمل `ConsumerStatefulWidget`.

تُفتح من «ابدأ» في الترحيب، ومن بطاقة «طالب جديد» في شاشة اختيار الملف.

lib/features/onboarding/create_profile_draft.dart — مسودة الخطوة 1

lib/features/onboarding/create_profile_draft.dart

السطر 1

```
1   import 'package:flutter_riverpod/flutter_riverpod.dart';
```

ماذا يفعل؟ استيراد Riverpod.

لماذا بهذه الطريقة؟ المسودة تعيش في مزوّد.

الأسطر 3–9

```
3   /// ما جُمع في الخطوة 1 (الاسم والصف) ريثما تكتمل الخطوة 2.
4   class CreateProfileDraft {
5     const CreateProfileDraft({required this.name, required this.grade});
6
7     final String name;
8     final int grade;
9   }
```

ماذا يفعل؟ صنف `CreateProfileDraft` بحقلين: الاسم والصف.

لماذا بهذه الطريقة؟ كائن بسيط غير قابل للتغيير يحمل ما جُمع في الخطوة 1.

الأسطر 11–13

```
11  class CreateProfileDraftNotifier extends Notifier<CreateProfileDraft?> {
12    @override
13    CreateProfileDraft? build() => null;
```

ماذا يفعل؟ `CreateProfileDraftNotifier` يرث `Notifier` وحالته `CreateProfileDraft?`؛ وقيمتها الأولى `null`.

لماذا بهذه الطريقة؟ `null` = لا مسودة (لم يبدأ الإنشاء).

الأسطر 15–17

```
15    void save({required String name, required int grade}) {
16      state = CreateProfileDraft(name: name, grade: grade);
17    }
```

ماذا يفعل؟ `save` : يحفظ مسودة جديدة بالاسم والصف.

لماذا بهذه الطريقة؟ تُستدعى عند الضغط على «متابعة».

الأسطر 19–20

```
19    void clear() => state = null;
20  }
```

ماذا يفعل؟ `clear` : يمسح المسودة ثم إغلاق الصنف.

لماذا بهذه الطريقة؟ بعد إنشاء الملف بنجاح.

الأسطر 22–25

```
22  final createProfileDraftProvider =
23      NotifierProvider<CreateProfileDraftNotifier, CreateProfileDraft?>(
24        CreateProfileDraftNotifier.new,
25      );
```

ماذا يفعل؟ `createProfileDraftProvider` : تسجيل المزوّد.

لماذا بهذه الطريقة؟ استعملنا مزوّدًا بدل تمرير بيانات عبر الراوتر: المسودة تبقى عند الرجوع من الخطوة 2 إلى 1، وأسهل في الاختبار، والراوتر يقرأها في `redirect` .

lib/features/onboarding/create_profile_step1_screen.dart — شاشة الخطوة 1

`lib/features/onboarding/create_profile_step1_screen.dart`

الأسطر 1–3

```
1   import 'package:flutter/material.dart';
2   import 'package:flutter_riverpod/flutter_riverpod.dart';
3   import 'package:go_router/go_router.dart';
```

ماذا يفعل؟ استيراد Material وRiverpod وgo_router.

لماذا بهذه الطريقة؟ الشاشة تقرأ المزوّدات وتتنقّل.

الأسطر 5–13

```dart
import '../../app/routes.dart';
import '../../core/constants/app_assets.dart';
import '../../core/strings/ar.dart';
import '../../core/theme/app_colors.dart';
import '../../core/theme/app_text_styles.dart';
import '../../core/widgets/mascot.dart';
import '../../core/widgets/primary_button.dart';
import '../../core/widgets/screen_top_bar.dart';
import 'create_profile_draft.dart';
```

ماذا يفعل؟ استيراد المسارات والأصول والنصوص والألوان والأنماط والشخصية والزر والشريط العلوي والمسودة.

لماذا بهذه الطريقة؟ كلها مكوّنات مشتركة؛ لا نعيد رسم أي شيء.

الأسطر 15–25

```dart
class _GradeGroup {
  const _GradeGroup({
    required this.title,
    required this.hint,
    required this.grades,
  });

  final String title;
  final String hint;
  final List<int> grades;
}
```

ماذا يفعل؟ صنف خاص `_GradeGroup` يصف مجموعة صفوف: العنوان والتلميح وقائمة الأرقام.

لماذا بهذه الطريقة؟ نحوّل المجموعات الثلاث إلى **بيانات** ونولّد الواجهة منها بحلقة بدل كتابتها ثلاث مرات.

الأسطر 27–43

```dart
const _groups = [
  _GradeGroup(
    title: ArStrings.gradeGroup14,
    hint: ArStrings.gradeGroup14Hint,
    grades: [1, 2, 3, 4],
  ),
  _GradeGroup(
    title: ArStrings.gradeGroup58,
    hint: ArStrings.gradeGroup58Hint,
    grades: [5, 6, 7, 8],
  ),
  _GradeGroup(
    title: ArStrings.gradeGroup912,
    hint: ArStrings.gradeGroup912Hint,
    grades: [9, 10, 11, 12],
  ),
];
```

ماذا يفعل؟ الثابت `_groups`: الصفوف 1–4 و5–8 و9–12 بنصوصها من `ArStrings`.

لماذا بهذه الطريقة؟ أرقام كل مجموعة تصاعدية: في RTL يظهر الأول يمينًا فيكون «1» أقصى اليمين كما في Figma.

الأسطر 45–52

```
/// الشاشة 02 – إنشاء الملف، الخطوة 1 من 2: الاسم والصف.
class CreateProfileStep1Screen extends ConsumerStatefulWidget {
  const CreateProfileStep1Screen({super.key});

  @override
  ConsumerState<CreateProfileStep1Screen> createState() =>
      _CreateProfileStep1ScreenState();
}
```

ماذا يفعل؟ تعليق وصنف `CreateProfileStep1Screen` من نوع `ConsumerStatefulWidget` و `createState`.

لماذا بهذه الطريقة؟ نحتاج حالة (الاسم والصف المختار) وقراءة مزوّدات، فنجمع الاثنين.

الأسطر 54–55

```
class _CreateProfileStep1ScreenState
    extends ConsumerState<CreateProfileStep1Screen> {
```

ماذا يفعل؟ صنف الحالة `_CreateProfileStep1ScreenState` من `ConsumerState`.

لماذا بهذه الطريقة؟ يعطي `ref` داخل الحالة.

الأسطر 56–57

```
  final _nameController = TextEditingController();
  int? _selectedGrade;
```

ماذا يفعل؟ الحقلان: `_nameController` للتحكم بنص الاسم و`_selectedGrade` (`int?`).

لماذا بهذه الطريقة؟ `TextEditingController` يعطينا قراءة النص والاستماع لتغيّره. `null` = لم يُختر صف.

الأسطر 59–66

```
  @override
  void initState() {
    super.initState();
    final draft = ref.read(createProfileDraftProvider);
    if (draft != null) {
      _nameController.text = draft.name;
      _selectedGrade = draft.grade;
    }
```

ماذا يفعل؟ `initState`: نقرأ المسودة؛ إن وُجدت نعبّئ الحقلين منها.

لماذا بهذه الطريقة؟ حين يرجع الطالب من الخطوة 2 تبقى اختياراته.

الأسطر 67–68

```
    _nameController.addListener(() => setState(() {}));
  }
```

ماذا يفعل؟ نضيف مستمعًا يستدعي `setState` عند كل تغيّر في الاسم.

لماذا بهذه الطريقة؟ الزر يتفعّل ويتعطّل حسب الاسم فيجب إعادة البناء مع كل حرف.

الأسطر 70–74

```
70    @override
71    void dispose() {
72      _nameController.dispose();
73      super.dispose();
74    }
```

ماذا يفعل؟ `dispose`: نتخلّص من الـ controller ثم `super.dispose()`.

لماذا بهذه الطريقة؟ منعًا لتسريب الذاكرة.

الأسطر 76–77

```
76    bool get _canContinue =>
77        _selectedGrade != null && _nameController.text.trim().isNotEmpty;
```

ماذا يفعل؟ `_canContinue`: يتحقق أن الصف مختار والاسم بعد `trim()` غير فارغ.

لماذا بهذه الطريقة؟ `trim` يتجاهل المسافات فلا يُقبل اسم من مسافات.

الأسطر 79–84

```
79    void _continue() {
80      ref
81          .read(createProfileDraftProvider.notifier)
82          .save(name: _nameController.text.trim(), grade: _selectedGrade!);
83      context.push(AppRoutes.createStep2);
84    }
```

ماذا يفعل؟ `_continue`: يحفظ الاسم والصف في المسودة ثم ينتقل للخطوة 2 بـ `push`.

لماذا بهذه الطريقة؟ `_selectedGrade!` آمنة لأن الزر لا يعمل إلا بعد `_canContinue`. `push` ليعود الطالب بزر الرجوع.

الأسطر 86–88

```
86    @override
87    Widget build(BuildContext context) {
88      return Scaffold(
```

ماذا يفعل؟ `build` وبداية `Scaffold`.

لماذا بهذه الطريقة؟ الهيكل الأساسي.

الأسطر 89–93

```
89        body: SafeArea(
90          child: SingleChildScrollView(
91            child: Column(
92              crossAxisAlignment: CrossAxisAlignment.stretch,
93              children: [
```

ماذا يفعل؟ `SafeArea` ثم `SingleChildScrollView` ثم `Column` ممتد.

لماذا بهذه الطريقة؟ التمرير ضروري: عند ظهور لوحة المفاتيح تضيق المساحة.

الأسطر 94–101

```
            ScreenTopBar(
              subtitle: ArStrings.gradeStep,
              title: ArStrings.gradeTitle,
              onBack: () => context.canPop()
                  ? context.pop()
                  : context.go(AppRoutes.welcome),
              end: const Mascot(size: 56),
            ),
```

ماذا يفعل؟ الشريط العلوي المشترك: «خطوة 1 من 2» و«اختر صفّك» وزر رجوع، والشخصية الصغيرة (56) في الطرف الآخر.

لماذا بهذه الطريقة؟ الرجوع: `pop` إن وُجدت شاشة سابقة وإلا نذهب للترحيب.

الأسطر 102–106

```
            Padding(
              padding: const EdgeInsetsDirectional.fromSTEB(20, 12, 20, 24),
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.stretch,
                spacing: 20,
```

ماذا يفعل؟ حشوة 20/12/20/24 و`Column` بمسافة 20.

لماذا بهذه الطريقة؟ قيم Figma.

الأسطر 107–108

```
                children: [
                  _NameField(controller: _nameController),
```

ماذا يفعل؟ بداية قائمة الأبناء وحقل الاسم `_NameField` بالـ controller.

لماذا بهذه الطريقة؟ نمرّر الـ controller الذي تملكه الحالة.

الأسطر 109–115

```
                  for (final group in _groups)
                    _GradeGroupSection(
                      group: group,
                      selected: _selectedGrade,
                      onSelected: (grade) =>
                          setState(() => _selectedGrade = grade),
                    ),
```

ماذا يفعل؟ حلقة تبني قسمًا لكل مجموعة وتمرّر الصف المختار ودالة الاختيار.

لماذا بهذه الطريقة؟ الاختيار يستدعي `setState` فتُظلَّل البلاطة. الحالة تعيش في الشاشة وتنزل للأبناء معاملات.

الأسطر 116–119

```
                    Text(
                      ArStrings.gradeChangeLater,
                      style: AppTextStyles.caption13,
                    ),
```

ماذا يفعل؟ ملاحظة «يمكنك تغيير الصف لاحقًا من حسابك.».

لماذا بهذه الطريقة؟ نص تطميني من Figma.

الأسطر 120–126

```
                    PrimaryButton(
                      label: _selectedGrade == null
                          ? ArStrings.continueLabel
                          : ArStrings.continueWithGrade(_selectedGrade!),
                      iconAsset: AppAssets.fwdGrade,
                      onPressed: _canContinue ? _continue : null,
                    ),
```

ماذا يفعل؟ زر المتابعة: «متابعة» قبل الاختيار و«متابعة · الصف N» بعده؛ وأيقونة السهم؛ والإجراء `_continue` أو `null`.

لماذا بهذه الطريقة؟ تمرير `null` يعطّل الزر (رمادي). النص الديناميكي هو ما يعرضه Figma.

الأسطر 127–136

```
                  ],
                ),
              ),
            ],
          ),
        ),
      ),
    );
  }
}
```

ماذا يفعل؟ إغلاق الأقواس والدالة والصنف.

لماذا بهذه الطريقة؟ نهاية `build`.

الأسطر 138–141

```
class _NameField extends StatelessWidget {
  const _NameField({required this.controller});

  final TextEditingController controller;
```

ماذا يفعل؟ ودجت `_NameField` ومعامل الـ controller.

لماذا بهذه الطريقة؟ يغلّف مظهر الحقل.

الأسطر 143–148

```
143   @override
144   Widget build(BuildContext context) {
145     const border = OutlineInputBorder(
146       borderRadius: BorderRadius.all(Radius.circular(16)),
147       borderSide: BorderSide(color: AppColors.line, width: 2),
148     );
```

ماذا يفعل؟ `build` وثابت محلي `border` : مستطيل بزوايا 16 وحد بسُمك 2 بلون `line`.

لماذا بهذه الطريقة؟ نعرّفه مرة ونعيد استعماله للحالات الثلاث (عادي ومفعّل وفي التركيز).

الأسطر 150–153

```
150     return Column(
151       crossAxisAlignment: CrossAxisAlignment.stretch,
152       spacing: 6,
153       children: [
```

ماذا يفعل؟ `Column` ممتد بمسافة 6.

لماذا بهذه الطريقة؟ بين العنوان والحقل.

السطر 154

```
154         Text(ArStrings.nameLabel, style: AppTextStyles.body16Strong),
```

ماذا يفعل؟ عنوان الحقل «ما اسمك؟».

لماذا بهذه الطريقة؟ نص مستقل فوق الحقل.

الأسطر 155–156

```
155         SizedBox(
156           height: 52,
```

ماذا يفعل؟ `SizedBox` بارتفاع 52 يحيط بالحقل.

لماذا بهذه الطريقة؟ يثبّت الارتفاع كما في Figma.

الأسطر 157–162

```
157           child: TextField(
158             controller: controller,
159             style: AppTextStyles.input17,
160             textInputAction: TextInputAction.done,
161             textCapitalization: TextCapitalization.words,
162             cursorColor: AppColors.teal,
```

ماذا يفعل؟ `TextField` بالـ controller ونمط `input17` وإجراء لوحة المفاتيح «تم» وأحرف أولى كبيرة ولون مؤشر teal.

لماذا بهذه الطريقة؟ اتجاه النص يُستنتج من اللغة (RTL) فيبدأ الكتابة من اليمين.

الأسطر 163–167

```
            decoration: InputDecoration(
              hintText: ArStrings.nameHint,
              hintStyle: AppTextStyles.input17.copyWith(
                color: AppColors.disabled,
              ),
```

ماذا يفعل؟ زخرفة الحقل: نص إرشادي «اكتب اسمك» باللون الرمادي.

لماذا بهذه الطريقة؟ إضافة غير موجودة في Figma لكنها لازمة لحقل يبدأ فارغًا.

الأسطر 168–170

```
              filled: true,
              fillColor: AppColors.card,
              contentPadding: const EdgeInsets.symmetric(horizontal: 16),
```

ماذا يفعل؟ خلفية بيضاء مملوءة وحشوة أفقية 16.

لماذا بهذه الطريقة؟ مظهر Figma.

الأسطر 171–172

```
              border: border,
              enabledBorder: border,
```

ماذا يفعل؟ نفس الحد للحالة العادية والمفعّلة.

لماذا بهذه الطريقة؟ يلغي حدود Material الافتراضية.

الأسطر 173–175

```
              focusedBorder: border.copyWith(
                borderSide: const BorderSide(color: AppColors.teal, width: 2),
              ),
```

ماذا يفعل؟ عند التركيز يتحوّل الحد إلى teal.

لماذا بهذه الطريقة؟ تغذية بصرية؛ `copyWith` تحافظ على نصف القطر.

الأسطر 176–182

```
            ),
          ),
        ),
      ],
    );
  }
}
```

ماذا يفعل؟ إغلاق الأقواس.

لماذا بهذه الطريقة؟ نهاية `_NameField`.

الأسطر 184–189

```
class _GradeGroupSection extends StatelessWidget {
  const _GradeGroupSection({
    required this.group,
    required this.selected,
    required this.onSelected,
  });
```

ماذا يفعل؟ ودجت `_GradeGroupSection` ومعاملاته.

لماذا بهذه الطريقة؟ قسم واحد (عنوان + 4 بلاطات) يتكرر 3 مرات.

الأسطر 191–193

```
  final _GradeGroup group;
  final int? selected;
  final ValueChanged<int> onSelected;
```

ماذا يفعل؟ الحقول: المجموعة والمختار و `ValueChanged<int>` .

لماذا بهذه الطريقة؟ الدالة تستقبل رقم الصف المختار.

الأسطر 195–200

```
  @override
  Widget build(BuildContext context) {
    return Column(
      crossAxisAlignment: CrossAxisAlignment.stretch,
      spacing: 8,
      children: [
```

ماذا يفعل؟ `build` و `Column` ممتد بمسافة 8.

لماذا بهذه الطريقة؟ بين العنوان وصف البلاطات.

الأسطر 201–209

```
        // Wrap: على شاشة ضيقة أو بخط مكبّر ينزل التلميح إلى السطر التالي بدل أن يفيض.
        Wrap(
          crossAxisAlignment: WrapCrossAlignment.end,
          spacing: 8,
          children: [
            Text(group.title, style: AppTextStyles.section18),
            Text(group.hint, style: AppTextStyles.caption13),
          ],
        ),
```

ماذا يفعل؟ تعليق و `Wrap` للعنوان «الصفوف 1–4» ثم التلميح، بمحاذاة `end` .

لماذا بهذه الطريقة؟ `Wrap` بدل `Row` : على شاشة ضيقة أو بخط مكبّر كان العنوان والتلميح يفيضان (17 بكسلًا على 320، 61 بخط ×1.5). الآن ينزل التلميح لسطر ثانٍ عند الحاجة. وضعنا `WrapCrossAlignment.end` بدل خط الأساس لأن `Wrap` لا يدعمه، والفرق بين الحجمين 18 و13 بسيط.

الأسطر 210–222

```
        Row(
          spacing: 10,
          children: [
            for (final grade in group.grades)
              Expanded(
                child: _GradeTile(
                  grade: grade,
                  selected: grade == selected,
                  onTap: () => onSelected(grade),
                ),
              ),
          ],
        ),
```

ماذا يفعل؟ صف البلاطات: لكل رقم `Expanded` يحوي `_GradeTile` بمسافة 10.

لماذا بهذه الطريقة؟ `Expanded` يوزّع العرض بالتساوي فتتكيّف مع كل الأعراض بدل عرض ثابت 80.

الأسطر 223–226

```
      ],
    );
  }
}
```

ماذا يفعل؟ إغلاق الأقواس.

لماذا بهذه الطريقة؟ نهاية الصنف.

الأسطر 228–233

```
class _GradeTile extends StatelessWidget {
  const _GradeTile({
    required this.grade,
    required this.selected,
    required this.onTap,
  });
```

ماذا يفعل؟ ودجت `_GradeTile` ومعاملاته.

لماذا بهذه الطريقة؟ بلاطة واحدة.

الأسطر 235–237

```
  final int grade;
  final bool selected;
  final VoidCallback onTap;
```

ماذا يفعل؟ الحقول الثلاثة.

لماذا بهذه الطريقة؟ غير قابلة للتغيير.

الأسطر 239–241

```
239   @override
240   Widget build(BuildContext context) {
241     final borderColor = selected ? AppColors.teal : AppColors.line;
```

ماذا يفعل؟ `build` ويحسب لون الحد: teal للمختار وإلا `line`.

لماذا بهذه الطريقة؟ نحسبه مرة ونستعمله لاحقًا.

الأسطر 243–247

```
243     return Semantics(
244       button: true,
245       selected: selected,
246       label: ArStrings.gradeSemantics(grade),
247       excludeSemantics: true,
```

ماذا يفعل؟ `Semantics` تعلن زرًّا محدَّدًا أو لا وتقرأ «الصف N».

لماذا بهذه الطريقة؟ قارئ الشاشة يقول «الصف 5، محدَّد».

الأسطر 248–250

```
248       child: AnimatedContainer(
249         duration: const Duration(milliseconds: 150),
250         height: 64,
```

ماذا يفعل؟ `AnimatedContainer` مدة 150 مللي ثانية وارتفاع 64.

لماذا بهذه الطريقة؟ تغيّر اللون يصير انتقالًا ناعمًا.

الأسطر 251–255

```
251         decoration: BoxDecoration(
252           color: selected ? AppColors.teal : AppColors.card,
253           borderRadius: BorderRadius.circular(18),
254           border: Border.all(color: borderColor, width: 2),
255         ),
```

ماذا يفعل؟ خلفية teal إن اختيرت وإلا بيضاء، زوايا 18، حد بسُمك 2.

لماذا بهذه الطريقة؟ مطابق لـ Figma.

الأسطر 256–260

```
256         child: Material(
257           type: MaterialType.transparency,
258           child: InkWell(
259             borderRadius: BorderRadius.circular(16),
260             onTap: onTap,
```

ماذا يفعل؟ `Material` شفاف و `InkWell` بزوايا 16 يلتقط الضغط.

لماذا بهذه الطريقة؟ الخلفية الملوّنة مرسومة في `AnimatedContainer` فنضع `Material` شفافًا فوقها ليظهر التموّج. نصف القطر 16 أقل بـ 2 من 18 بسبب سُمك الحد.

الأسطر 261–270

```
            child: Center(
              child: Text(
                '$grade',
                style: AppTextStyles.gradeNumber24.copyWith(
                  color: selected ? AppColors.card : AppColors.ink,
                ),
              ),
            ),
          ),
        ),
```

ماذا يفعل؟ الرقم في المنتصف بنمط `gradeNumber24`، أبيض عند الاختيار وداكن عند غيره.

لماذا بهذه الطريقة؟ تباين عالٍ في الحالتين.

الأسطر 271–274

```
      ),
    );
  }
}
```

ماذا يفعل؟ إغلاق الأقواس.

لماذا بهذه الطريقة؟ نهاية الملف.